

Речь на защите Duekeep

Ориентир: **8–9 минут** спокойным темпом (примерно 1100–1200 слов). Если комиссия смотрит экран — после блока про продукт можно на 30–40 секунд показать UI, не растягивая.

Читать вслух. В скобках — паузы и ремарки, не произносить.

Добрый день. Меня зовут Сергей Фищук. Тема работы — веб-приложение **Duekeep**: календарь обязательств. Это full-stack система, которая отвечает на простой вопрос: *что надо оплатить в ближайшие дни и сколько это будет стоить*.

Коротко о результате, чтобы было понятно, о чём речь. После клонирования репозитория проект поднимается командой `docker compose up`. Есть живой стенд на duekeep.ru. Локально — тестовый пользователь и демо-каталог. На проде демо-seed выключен: каждый регистрируется и видит только свои данные.

(пауза)

Зачем это нужно. У нас у всех в современной жизни накопилось множество обязательств по оплате: облачные сервисы, домены, подписки, стриминги, аренда, страховка, налог, телефоны, интернет и тп. В какой-то момент я понял что не знаю точно кому и за что я плачу или должен заплатить. Duekeep по сути - *платежный календарь*, считает статус по дате, показывает деньги и умеет напоминать о наступающем или прошедшем сроке в ленте или в пуше если приложение установлено как PWA.

Что умеет программа. Пользователь регистрируется или входит, ведёт свои записи: тип, категория, сумма, валюта, период оплаты, срок, порог уведомления. Есть не детерминированные справочники, Специфика типа — VIN, регистратор, номер полиса — лежит в JSONB, а срок и деньги остаются колонками, чтобы фильтры и дашборд не ломались.

Статус записи меняется или автоматически или руками. Тикер при старте сервера и далее раз в двенадцать часов пересчитывает «активно / скоро истечёт / истекло» по календарному дню. Можно заморозить всю запись как «оплачено», можно сказать «не уведомлять». Отдельно — оплата конкретного вхождения: подписка за сентябрь и за октябрь это разные факты. В календаре день остаётся, меняется статус вхождения. С карточки и с обзора можно оплатить ближайший открытый срок, не ходя в сетку.

Есть дашборд: сколько активных и просроченных, суммы по месяцам, разрез по валюте. Есть список с фильтрами, карточка, продление с историей, журнал аудита, экспорт и импорт CSV. Уведомления в ленте. Web Push в ОС. Клиент — PWA: можно поставить на домашний экран.

Данные изолированы. Программа многопользовательская. На проде нет демо-админа с пятьюдесятью записями.

(пауза)

Стек сознательно простой, без лишних слоёв. Backend — Go 1.25, один бинарь, слои handler, service, repository. HTTP — chi, база — PostgreSQL и pgx, миграции — goose. Авторизация — JWT

access и refresh с ротацией семейства токенов. Контракт API описан в OpenAPI, рядом живой Swagger. Frontend — React, TypeScript, Vite, Tailwind, PWA. В разработке проект управляется через Таскфайл: `task lint`, `task test`. Деплой: GitHub Actions после зелёного CI включаетс CD по SSH собирает прод-compose на VPS. Секреты только в `.env` на сервере, не в git.

Нет никаких микросервисов, оркестрации, нод, шардирования и тп.

Архитектуру мы зафиксировали до большого объёма кода: сначала функционал, потом схема и границы «что не делаем». Не делали организации, шаринг, почту, вложения, офлайн-редактирование. Это не недоделка «не успели», а рамка диплома: довести вертикальный срез до работающего продукта.

Теперь — как это сделано. Это важная часть защиты.

Проект написан не в одиночку в редакторе «с нуля по учебнику». **Основной объём кода, тестов и документации появился в диалоге с агентом в Cursor.** Модель, с которой я работал чаще всего, — **Cursor Grok**. Я не прячу этот факт: наоборот, считаю его предметом работы. Вопрос не в том, «умеет ли нейросеть генерировать файлы». Вопрос в том, *кто задаёт цель, кто режет scope, кто проверяет результат и кто отвечает за инварианты.*

Моя роль была ролью постановщика, ревьюера и конечного тестировщика.

Первый шаг — в чат ушли требования к диплому. Затем я зафиксировал функционал в **FUNCTIONAL.md** и архитектуру в **ARCHITECTURE.md**. Код сознательно не начинали, пока не закрыли спорные решения: что в колонках, что в JSONB, какой auth, какой realtime.

Весь предполагаемы объём работ **разделили на спринты и дальше работа шла спринтами. У каждого спринта четыре документа:**

- план,
- чеклист,
- контракт API,
- список ограничений. Правило простое: *агент не начинает спринт сам и не выдумывает функционал*. Сменил ручку — обнови спеку. Пункт чеклиста не отмечается без проверки. Журнал **REPORT.md** пишется по ходу, не «в конце из головы».

Чтобы агент не забывал договорённости между чатами, в репозитории лежат правила и скилл Dueкеер. Там слои, изоляция по `owner_id`, запрет тащить Redis и второй инстанс, команда линтера. Я их не игнорировал: **без этого модель через три диалога начинает «улучшать» архитектуру**. Для экономии контекста новый чат я часто начинал с *субагента*, который читает текущий чеклист и код — не устаревший абзац в старом документе.

Как выглядел типичный цикл.

- Агент читает документацию и запускает субагента для определения текущего состояния
- Агент выполняет пункт спринта прогоняет линтеры и тесты и ждет подтверждения
- Я тестирую или свагер или UI если он уже есть, вношу коррективы или даю подтверждение
- Агент начинает следующий пункт

- Я формулирую задачу на языке продукта и ограничений: «нужна оплата дня в календаре, не меняй статус всей записи, не ломай прод». Агент предлагает вариант. Я смотрю, не вылезает ли он за спринт. Если вылезает — отсекаю. Если вариант годный — разрешаю писать код. Потом гоняю lint и тесты, смотрю UI, на проде проверяю, что демо-seed не включился. Коммит и деплой — только когда я об этом прошу.

Что я сознательно не отдавал агенту. Секреты прода. Делать деплой на без зелёного CI. Решение готов ли пункт к галочке. Если я сам не открыл экран и не прогнал сценарий — DoD не закрыт.

Итог по методу. ИИ хорошо пишет однородный код, тесты, спеку и черновики документов, если есть жесткие рамки и хорошо документированный код. Плохо — если рамок нет: расползается score, путает вчерашнюю модель «общий каталог» с сегодняшней «свои данные», чинит симптом вместо причины. Поэтому артефакты процесса — рулы, скиллы, спринты, limitations — это важный способ сделать генерацию управляемой.

Собственно если весь проект разделить по времени:

- 30% - утрясание архитектуры и функционала
- 20% - организация проекта - скиллы, рулы, субагенты
- 20% - деплой
- 30% - кодинг

Что можно показать комиссии за минуту, если будет экран. Клон и docker compose up. Вход admin@duekeep.local. Дашборд, календарь с оплаченным днём, карточка, колокольчик, /docs. Регистрация второго пользователя — чужих записей нет. Прод: тот же продукт, без демо-аккаунтов.

Спасибо. Готов ответить на вопросы — и по предметной области, и по тому, как именно ставились задачи агенту.